

Homework 5

CS 521: Fall 2025

Due on Friday, December 12, 2025 11:59 PM Central

Homework Policy: You are allowed to collaborate with your classmates, but report this in your submission. Even if you work on problems together, each student must write up their solution individually in their own words in their submission. This CS 473 course page on academic integrity is a handy reference: <https://courses.engr.illinois.edu/cs473/sp2023/integrity.html>. We have ZERO tolerance for breaches of academic integrity, such as, but not restricted to, plagiarism, use of LLMs except for when stated explicitly, etc.

You can take up to 3 days extra in total across the semester without explanation. Once you exhaust the three-day late submission allowance, any further late homework submissions will result in a 25% penalty per each extra day. 5 minutes late or more in submitting the homework will be counted as 1 day late. If you have exceptional personal circumstances that will prevent you from submitting the assignment on time, write to the course staff as soon as possible. Unless explicitly stated, use of LLMs for solving homework or cheating from others is not allowed – we will detect it! There will be an oral **exam** if we detect violations, during which you will be asked to answer a randomly selected question from the homework. If you are unable to show sufficient knowledge, then you will get zero for the full homework.

Please format your submission as a PDF (file naming convention: *CS521-⟨netid⟩_hw5.pdf*) and upload it on Gradescope by the deadline. Try to start every problem on a new page, to accurately map pages to questions on Gradescope.

Programming Problems: **Read these guidelines carefully!** Describe your solution at a high-level (if applicable). Write only small snippets of code, if at all. DO NOT paste tall walls of code into your submission! Upload your solution files to GitHub publicly and provide a link to the relevant files as part of your solution.

Points: Problem 1: $10 + 10 + 15 = 35$ points, Problem 2: 35 points, Problem 3: $5 + 10 + 15 = 30$ points. Total: 100 points

Problem 1 (Observing SuperPosition). As discussed in the lecture, it is believed that deep neural networks can represent more features than they have neurons. The paper

<https://arxiv.org/pdf/2209.10652> discusses how to empirically demonstrate feature superposition, focusing on toyish learning tasks.

- (a) (10 points) Read the paper and explain if their experimental design is suitable for showing superposition (intuition is fine, formal details are even better).
- (b) (10 points) Next, run their code, available at https://colab.research.google.com/github/anthropics/toy-models-of-superposition/blob/main/toy_models.ipynb to reproduce their experimental results. Describe if the results show superposition and why or why not? (Make sure to attach the plots from running their code to your solution when discussing your observations).
- (c) (15 points) Extend the experiments to at least one more non-toy task (e.g., MNIST classification) using a larger (more layers or neurons) network; analyze whether you observe superposition.

Problem 2 (Syncode, programming). You are given a mathematical problem involving binary operations (addition, subtraction, multiplication, or division). Your task is to:

1. Use SynCode to extract the two floating point operands and the (single) operator from the query.
2. Perform the operation using the extracted values using Python interpreter.

Examples

Query: What is 327. multiplied by 11.0?

SynCode will extract the operands and the operator:

- Operands: 327.0 and 11.0
- Operator: *

The Python program can then calculate $327.0 * 11.0$

Final Answer: 3597.0

Query: What is 45.1 plus 23.54?

Final Answer: 68.64

Query: What is 120.4 divided by 4.0?

Final Answer: 30.1

Important notes. You can use the official implementation of Syncode <https://github.com/structuredllm/syncode>, to develop your solution. In addition to providing the grammar for SynCode, it is essential to include 3-4 few-shot examples in the prompt. These examples should demonstrate how the model extracts the operator and operands from the query. The examples should focus solely on the extraction process and should not include the final result (computed using an external tool, such as the Python interpreter). This part of the prompt can be reused for all queries and should be placed before the user query to guide the model in correctly identifying the operator and operands.

Solution requirements. You must use Microsoft's Phi-2 model (<https://huggingface.co/microsoft/phi-2>) within Syncode. Your solution must be provided as a self-contained Google Colab notebook without any dependencies on external files. In particular, you must define a function `run_syncode(textual_query)` that should take the textual query as input and run Syncode to return a tuple (ordered list of operands, operator as a string, final answer as float) in the mentioned order. We will run automated tests on this function in your code. You can define other helper functions or data structures, but we will not use them directly. Not adhering to the above guidelines will result in 0 points, as we will not be able to run your code. We will grade your solution as (% test cases passed * total points). The test cases will be similar to the above examples in their design. Hence, you can test your implementation with the above examples.

Hints. You can look at the example notebooks provided along with Syncode's official implementation here <https://github.com/structuredllm/syncode/tree/main/notebooks> to understand its usage.

Problem 3 (Activation Steering). The paper <https://arxiv.org/abs/2406.11717> shows that a single direction is responsible for triggering refusal behavior in large language models.

- (a) (5 points) Read the paper and explain their method for finding the refusal direction and showing its necessity and sufficiency using activation addition and directional ablation.
- (b) (10 points) Run experiments to reproduce their methodology (finding the refusal direction, activation addition, and directional ablation on the datasets used in their experiments) for one open source language model of your choice. You can leverage the official implementation available at https://github.com/andyrdt/refusal_direction. Report your findings by drawing plots similar to figures 1 and 3 in the paper. Explain what you see.

- (c) (15 points) Choose a different behavior (e.g., “truthfulness” <https://arxiv.org/pdf/2304.13734>). Propose and implement a method (inspired by refusal-direction) to discover a candidate “behavior direction.” Test whether addition/ablation along that direction indeed affects the behavior. Report methodology, experiments, successes/failures, and detailed analysis.

Bonus Research Question (25 points). In the lecture, we discussed the IterGen <https://openreview.net/forum?id=ac93gRzxxV> method that backtracks when the partial generation violates a semantic constraint. The violation is fixed by sampling another response from a modified output distribution over tokens. In this task, your goal is to develop a method that leverages activation steering for fixing violation.

- (a) (5 points) Describe your approach at a high level (you can also provide pseudocode).
- (b) (20 points) Develop an implementation of your approach and evaluate on tasks considered in the IterGen paper. You can leverage the official implementation of IterGen available at <https://github.com/structuredllm/itergen>. Report your results.